

Layered Architecture Components of The Architecture

1. **Controller** : Handles user requests and responses.
2. **BO (Business Object)** : Contains business logic of the system.
3. **DAO (Data Access Object)** : Handles database operations (CRUD).
4. **Entity** : Represents database table structure. Database table එක represent කරන class එක.
5. **DTO (Data Transfer Object)** : Used to transfer data between layers safely. Layers අතර data safely transfer කරන්න use කරන object එක.

Controller – handles user requests

BO – business logic

DAO – database operations

Entity – database table mapping

DTO – transfers data between layers

DB – stores data

DAO (Data Access Object)

Sinhala: Database operations (save, update, delete) manage කරන්න. Business logic එකෙන් වෙනම.

DAOImpl

Sinhala: DAO interface එක implement කරන class එක. SQL queries ලියන එක.

CrudDAO

Sinhala: Save, update, delete, getAll වගේ common DB methods එක interface එකේ define කරනවා.

SuperDAO

Sinhala: DAO classes වලට common interface එක. එකම style එකේ methods තියෙනව කියලා confirm කරන්න.

BO (Business Object)

Sinhala: Business rules handle කරන class. DAO use කරලා data handle කරනවා.

BOImpl

Sinhala: BO interface එක implement කරන class. Real logic තියෙනවා.

Enum (DAOFactory / BOFactory)

Sinhala: Object create කරන්න convenient method. DAOFactory.getInstance() වගේ.

Polymorphism

Sinhala: එකම method එක different ways වලින් work කරන්න පුළුවන්.

Layers of Layered Architecture

1. Presentation Layer

- This is the UI (User Interface).
- Handles user input and output.

Example: JavaFX forms, controllers, web pages.

2. Business Layer (Service Layer)

- Contains the business logic.

- Processes data and applies rules.
- Connects Presentation Layer ↔ Persistence Layer.

3. Persistence Layer (DAO Layer)

- Handles database operations.
- Includes DAO classes.

Example: Save, update, delete, search data in DB.

4. Database Layer

- The actual database where data is stored.

Example: MySQL, PostgreSQL.

Flow:

User → Presentation → Business → Persistence → Database

Advantages of Layered Architecture

- **Clear structure** – system organized into layers (Controller, BO/Service, DAO).
- **Loose coupling** – layers are independent.

- **Easy maintenance** – changes in one layer do not affect others. එක layer එකක් වෙනස් කලත් අනිත් layers වලට බලපෑම ඇතිවේ.
- **Scalability** – suitable for large enterprise applications. Large enterprise applications වලට හොඳට use කරන්න පුළුවන.
- **Reusability** – business logic and data access can be reused. Business logic සහ data access code නැවත භාවිතා කරන්න පුළුවන.

Difference between Model–View–Controller (MVC) and Layered Architecture.

Feature	MVC Architecture	Layered Architecture
Structure	3 main components	Multiple layers
Main Parts	Model, View, Controller	Controller, Service/BO, DAO, Entity, DB
Purpose	Separate UI from business logic	Organize whole system into layers
UI Handling	View handles UI	Presentation layer handles UI
Business Logic	In Model	In Service / BO layer
Database Access	Model may access DB	DAO layer handles DB
Complexity	Simpler	More structured and scalable
Usage	Mostly web applications	Large enterprise applications

key software design principles (rules)

Loose Coupling

- **Purpose:** Make system flexible and easy to maintain.
- **Meaning:** Components depend less on each other.

- **What to do:** Use interfaces, avoid direct class dependencies.

Interfaces use කරන්න, direct class dependencies අඩු කරන්න.

- **Where to use:** Between layers (Controller → Service → DAO).

Dependency Injection

- **Purpose:** Reduce hard-coded dependencies and improve testability.

Hard-coded dependencies අඩු කරන්න සහ test කරන්න easy.

- **Meaning:** Objects are provided from outside, not created inside

Object provide කරනවා, class එකේ ඇතුළේ create නොකරන්න.

- **What to do:** Inject Service into Controller, DAO into Service.

Service එක Controller එකට object inject කරනවා, DAO එක Service එකට object inject කරනවා.

- **Where to use:** Layer connections in layered architecture

High Cohesion

- **Purpose:** Classes focused and easier to maintain.

- **Meaning:** Each class/module has one clear responsibility.

- **What to do:** Keep DAO for DB, Service for logic, Controller for UI only.

- **Where to use:** Every class in the project.

Less Boilerplate Code

- **Purpose:** Keep code clean and shorter.

- **Meaning:** Avoid repeated or unnecessary code.

Repeated සහ අනවශ්‍ය කේතය මඟ හැරීම

- **What to do:** Use ORM frameworks, utility methods, and inheritance.

- **Where to use:** DAO operations, Service methods, DB mappings.

boilerplate

public static → Object **අරගෙන නැතුවම method එක use කරන්න පුළුවන්.**

boolean → **true / false** return වෙනවා (කියන්නේ success කියලා).

execute → SQL query **run කරන method එක.**

String sql → Run කරන්න තියෙන **SQL query.**

Object... params → SQL එකට **pass කරන්න පුළුවන් multiple values.**

throws SQLException, ClassNotFoundException → **DB errors handle කරන්න.**

dependency injection

getBO(BOType type) → **මිනිසුන් BO object එකක් ලබාගන්න method එකක්.**

switch (type) → **type එක check කරලා** right object එක return කරලා දෙනවා.

return new GemBOImpl() → **GemBOImpl එකක් create කරලා return කරනවා.**

Dependency Injection කියන්නේ: client එකට **directly object create කරන්න එපා.** Factory එක object inject කරනවා. මේකෙන් **code එක loosely coupled වෙනවා.**

loose coupling

Factory එකෙන් object එක request කරනවා → getBO(BOType.CUSTOMER)

Interface type එකට cast කරනවා → (CustomerBO)

Loose Coupling කියන්නේ: code එක **implementation** එකට **depend** නොවනවා.

Benefit: පස්සේ CustomerBOImpl වෙනස් කරගන්න පුළුවන් **client code touch** නොකරලා.

1. Layered Architecture කියන්නේ?

System එක layers වලට බෙදලා responsibilities වෙන් කරන එක.

2. Layers මොනවාද?

Presentation, Business, Persistence, Data

3. Presentation Layer purpose?

User interface සහ requests handle කරන්න.

4. Business Layer purpose?

Business logic / rules තියෙන්න.

5. Persistence Layer purpose?

Database operations (CRUD) handle කරන්න.

6. Data Layer purpose?

Actual data store කරන්න (tables/entities).

7. Loose Coupling කියන්නේ?

Layers independent, එක වෙනස් වුණත් අනිත් layers break නොවෙයි.

8. Dependency Injection කියන්නේ?

Objects class ඇතුළින් create කරන්නෙ නැතුව, බාහිරින් provide කරන එක.

9. High Cohesion කියන්නේ?

එක class/module එකේ එක clear responsibility තියෙන්නෙ.

10. DTO use කරන්නෙ?

Layers අතර data safely transfer කරන්න.

11. Objects inject වෙනවා?

DAO → Service, Service → Controller

12. Layered Architecture rules?

Top-down calls only, layers skip කරන්න එපා, one responsibility per layer

13. Advantages?

Organized, maintainable, scalable, reusable

14. Flow of layers?

User → Controller → Service → DAO → Database

15. MVC vs Layered?

MVC → UI සහ logic separate කරනවා

Layered → Whole system layers වල organize කරනවා

16. ඇයි Layered Architecture project වල use කරන්නේ?

Maintainable, scalable, loose coupling improve කරන්න

17. Upper layer call කරන්න පුළුවන්ද?

නැහැ, only top-down අවසර දී ඇත.

18. Layer skip කරන්න පුළුවන්ද?

නැහැ, must call directly below layer

19. Presentation Layer example?

Controller class (GemController)

20. Business Layer example?

Service / BO class (GemServiceImpl)

21. Persistence Layer example?

DAO class (GemDAOImpl)

22. Data Layer example?

Entity class (Gem.java), database tables

23. Dependency Injection උපකාරී වන්නේ කෙසේද?

Coupling අඩු කරන්න, testing easy, flexible

24. DTO benefit?

Layers අතර data safely transfer, entities expose නොකර

25. ඇයි High cohesion important?

Maintenance easy, clarity, reusability

26. ඇයි Less boilerplate code good?

Code clean, short, readable

27. Flow example (Gem project)?

User → GemController → GemService → GemDAO → Database

28. සේවාවට DAO එන්නත් කිරීමේ වාසිය?

Service එක concrete DAO depend නොවෙයි, replace/test easy

29. පාලකයට සේවාව එන්නත් කිරීමේ වාසිය?

Controller only UI/requests handle කරනවා, logic handle නොකර

30. Multiple layers වලට same DTO බෙදා ගත හැකිද?

ඔව්, safely layers අතර data transfer කරන්න

